

Assistente Inteligente de Estudos

Arquitetura de microsserviços para tutoria acadêmica baseada em RAG (Retrieval-Augmented Generation), busca híbrida com reranking e modelo de linguagem de grande porte (LLM)

Sumário

1. Introdução e Visão Geral	3
1.1 Objetivos do Sistema	3
1.2 Tecnologias Empregadas	3
2. Arquitetura Geral	4
3. Descrição dos Componentes	5
3.1 Frontend (React / Vite / TypeScript)	5
3.2 API Gateway (Node.js / Express)	5
3.3 RAG Service (Python / FastAPI).....	5
3.4 IA / Proxy Service (Python / FastAPI).....	6
3.5 Histórico Service (Python / FastAPI / PostgreSQL).....	6
3.6 MCP Service (TypeScript / Express)	6
3.7 Ambiente de Nuvem — Qwen 14B no Google Colab.....	7
4. Fluxo de uma Requisição	7
5. Pipeline de Recuperação (RAG)	7
5.1 Indexação em Pipeline (4 estágios paralelos)	7
5.2 Recuperação Híbrida + Reranking	7
5.3 Roteamento Material × Web	8
5.4 Casos Especiais: Resumo, Questões e Identidade.....	8
6. Camada de Geração (LLM)	8
7. Decisões Técnicas e Lições Aprendidas	9
8. Limitações e Trabalhos Futuros	9
9. Conclusão	9

1. Introdução e Visão Geral

Este relatório descreve, em detalhe técnico, o sistema **Assistente Inteligente de Estudos** — carinhosamente apelidado de **ARIA**: um tutor virtual que lê o material de aula de um estudante (em formato PDF) e responde, em linguagem natural e em português do Brasil, a perguntas sobre esse conteúdo. Quando a resposta não está no material, o sistema recorre automaticamente a uma busca na web, sinalizando ao estudante a origem da informação. Ao longo deste documento, o assistente é referido pelo apelido **ARIA**, que é também o nome pelo qual ela se identifica nas conversas com os estudantes.

A solução foi construída sobre uma **arquitetura de microsserviços**: cada responsabilidade (orquestração, recuperação de contexto, geração de texto, histórico de conversa e ferramentas externas) é isolada em um serviço independente, com sua própria tecnologia e porta de rede. Essa separação favorece a manutenção, o teste isolado e a substituição de componentes sem impacto no restante do sistema.

1.1 Objetivos do Sistema

- Responder perguntas de estudantes com **fidelidade ao material** de aula (grounding), evitando alucinações e informações inventadas.
- Rotear automaticamente entre **material de aula** (PDFs) e **busca na web** conforme a relevância do conteúdo recuperado.
- Manter **memória de conversa** por sessão, dando continuidade ao diálogo.
- Oferecer recursos didáticos: **resumos** abrangentes do material e **geração de questões** (abertas e fechadas) para revisão.
- Garantir uma **identidade consistente** (a tutora se chama sempre ARIA), independentemente do conteúdo do PDF ou do comportamento do modelo de linguagem.

1.2 Tecnologias Empregadas

Camada	Tecnologia
Interface (frontend)	React 19, Vite, TypeScript, react-markdown
Orquestração	Node.js, Express, Axios
Recuperação (RAG)	Python, FastAPI, LlamaIndex, ChromaDB, BM25
Embeddings	intfloat/multilingual-e5-large (GPU/CUDA)
Reranking	BAAI/bge-reranker-base (cross-encoder, CPU)
Geração (LLM)	Qwen 2.5 14B via Ollama (Google Colab + túnel ngrok)
Histórico	Python, FastAPI, SQLAlchemy, PostgreSQL
Ferramentas externas (MCP)	TypeScript, Express, API Tavily (busca web)

2. Arquitetura Geral

O sistema é organizado em quatro camadas lógicas: **cliente** (frontend), **orquestração** (API Gateway), **microserviços** (RAG, IA/Proxy, Histórico e MCP) e **recursos externos** (banco de dados, base vetorial, API de busca e o LLM hospedado na nuvem). O estudante interage apenas com o frontend; toda a complexidade de roteamento, recuperação e geração fica encapsulada por trás do Gateway.

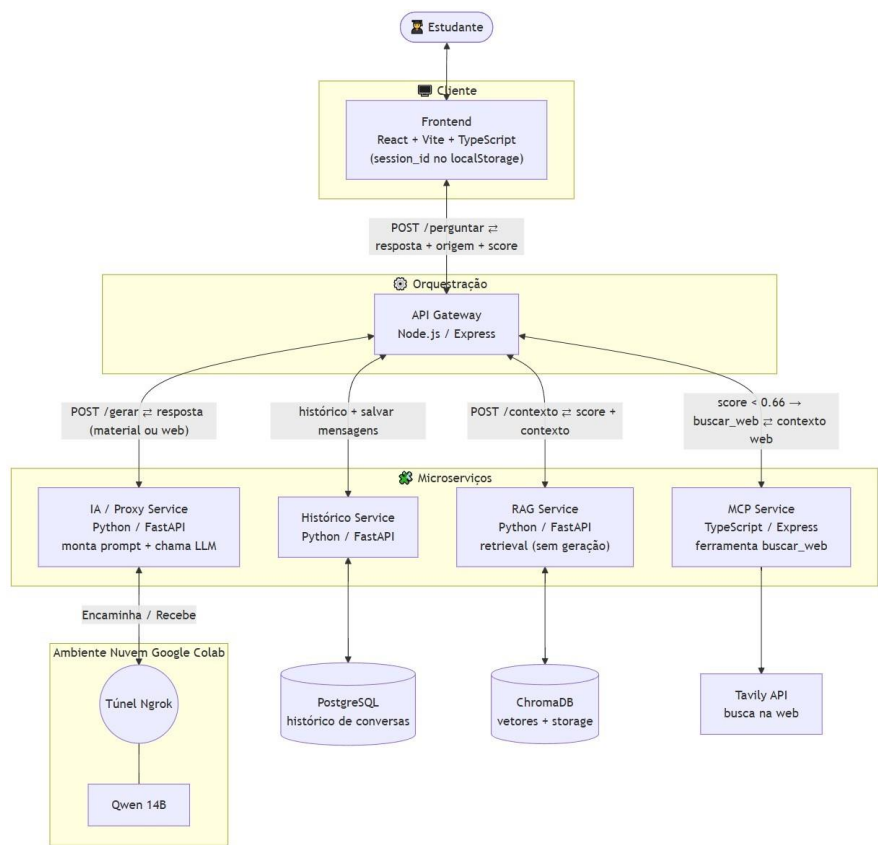


Figura 1 — Arquitetura de microserviços do sistema ARIA.

Cada serviço expõe uma API HTTP própria e roda em uma porta local distinta. A comunicação entre serviços é feita por chamadas HTTP (JSON), o que mantém o acoplamento baixo: qualquer serviço pode ser reiniciado ou substituído desde que respeite seu contrato de API.

Serviço	Tecnologia	Porta	Responsabilidade
Frontend	React/Vite/TS	5173	Interface de chat e upload de PDF
API Gateway	Node/Express	8000	Orquestração e roteamento
RAG Service	Python/FastAPI	8001	Indexação e recuperação de contexto
MCP Service	TypeScript/Express	8002	Ferramenta de busca na web (Tavily)
Histórico Service	Python/FastAPI	8003	Memória da conversa (PostgreSQL)
IA / Proxy Service	Python/FastAPI	8004	Montagem de prompt e geração (LLM)

3. Descrição dos Componentes

3.1 Frontend (React / Vite / TypeScript)

A interface é uma aplicação de página única (SPA) em React 19 com Vite. Apresenta um chat em que o estudante pode digitar perguntas e anexar um PDF de material. As respostas da ARIA são renderizadas em **Markdown** (via `react-markdown` + `remark-gfm`), com destaque em negrito, listas e tabelas.

- Cada navegador gera e persiste um **session_id** (UUID) no `localStorage`, usado para dar memória à conversa.
- As bolhas de mensagem são **memoizadas** (`React.memo`) para evitar reprocessar o Markdown de todo o histórico a cada tecla digitada.
- Uma função de formatação trata o caso recorrente de o modelo emitir alternativas de múltipla escolha (a, b, c, d) em uma única linha, quebrando-as em itens de lista.
- O upload de PDF e o envio de perguntas comunicam-se exclusivamente com o API Gateway (`http://127.0.0.1:8000`).

3.2 API Gateway (Node.js / Express)

O Gateway é o **cérebro de orquestração** do sistema: é o único ponto de contato do frontend e coordena todos os demais serviços. Para cada pergunta, ele executa um fluxo de decisão que envolve histórico, recuperação de contexto, roteamento e geração.

As principais responsabilidades são:

- **Memória:** busca as últimas mensagens da sessão no Histórico Service e as injeta na geração; salva tanto a pergunta quanto a resposta (de forma *best-effort* — se o histórico cair, a conversa continua).
- **Deteção de identidade:** reconhece perguntas sobre o nome da assistente (“qual seu nome?”). Se for pergunta pura de identidade, responde sem recuperação; se for combinada (nome + conteúdo), ativa um reforço de identidade.
- **Limpeza da consulta:** remove saudações, gentilezas e perguntas de identidade antes de enviar a consulta ao RAG, evitando que a busca recupere a ficha do documento em vez do conteúdo relevante.
- **Roteamento material × web:** decide a origem da resposta com base no score de relevância retornado pelo RAG (ver Seção 5.3).
- **Âncora temática:** quando o pedido se refere ao material mas cai na busca web, ancora a consulta no tema do PDF (título da ficha) para que os resultados não fujam do assunto.
- **Tratamento de erros:** traduz falhas do LLM (Colab/ngrok offline, túnel expirado) em mensagens acionáveis exibidas diretamente ao estudante.

3.3 RAG Service (Python / FastAPI)

O RAG Service é responsável **exclusivamente pela recuperação** (retrieval) — ele não gera texto. Indexa o PDF do estudante, mantém a base vetorial e, dado um texto de consulta, devolve os trechos mais relevantes acompanhados de um **score de relevância calibrado** que orienta o roteamento do Gateway. O pipeline de indexação e a estratégia de recuperação são detalhados na Seção 5.

- **Embeddings** com o modelo `multilingual-e5-large` na GPU (CUDA), oferecendo busca semântica multilíngue.

- **Base vetorial** ChromaDB persistente, com armazenamento dos *chunks* e seus vetores.
- **Busca híbrida** combinando recuperação vetorial e lexical (BM25) via fusão (*reciprocal rank fusion*).
- **Reranking** por um *cross-encoder* (bge-reranker-base) que reordena os candidatos por relevância real.
- **Política “um livro por vez”**: cada upload substitui o material anterior, evitando mistura de assuntos de PDFs diferentes.
- **Ficha do documento**: um *chunk* sintético com título e autor do PDF, robusto a metadados ruins, que responde perguntas sobre o próprio material.

3.4 IA / Proxy Service (Python / FastAPI)

Este é o **único serviço que conversa com o LLM** (Qwen 2.5 14B, hospedado no Google Colab e exposto via túnel ngrok). Ele recebe do Gateway um pedido estruturado (pergunta + contexto + histórico + fonte), seleciona o **template de prompt** adequado, monta o prompt final, chama o modelo e aplica pós-processamento determinístico à resposta. Não faz recuperação nem acessa banco de dados.

- **Três templates de prompt**: material de aula, busca na web e resumo (este último mais enxuto, para liberar espaço de saída dentro da janela de contexto do modelo).
- **Identidade determinística**: perguntas sobre o nome recebem resposta fixa (sem chamar o LLM), garantindo que a tutora seja sempre “ARIA” — o modelo tende a se dizer “Claude”.
- **Instruções para questões**: ao gerar exercícios, injeta regras de formato e anti-repetição; em seguida, remove marcadores Markdown desbalanceados e, se pedido, o gabarito.
- **Robustez**: traduz erros de conexão (403 do ngrok, timeout, Colab offline) em respostas HTTP 503 com mensagens claras.

3.5 Histórico Service (Python / FastAPI / PostgreSQL)

Persiste o histórico de conversas em PostgreSQL, no modelo normalizado “uma linha por mensagem”. Cada registro guarda o identificador da sessão, o papel (*usuario* ou *assistente*), o conteúdo e o instante de criação. O Gateway consulta as últimas mensagens para compor a memória e grava cada nova interação.

Método / Rota	Função
POST /mensagens	Salva uma mensagem da sessão
GET /historico/{sessao}	Recupera as últimas N mensagens (ordem cronológica)
DELETE /historico/{sessao}	Limpa o histórico de uma sessão
GET /health	Verifica a conexão com o banco

3.6 MCP Service (TypeScript / Express)

Implementa o padrão **MCP (Model Context Protocol)** sobre HTTP, expondo ferramentas externas ao restante do sistema. Atualmente oferece a ferramenta `buscar_web`, que consulta a **API Tavily** e devolve os resultados formatados como texto, prontos para o LLM sintetizar. O serviço é acionado pelo Gateway quando a resposta não está no material de aula.

3.7 Ambiente de Nuvem — Qwen 14B no Google Colab

O modelo de linguagem roda em uma instância do **Google Colab** com GPU, servido pelo **Ollama** e exposto à internet por um **túnel ngrok** com domínio fixo. A sequência de inicialização instala as dependências, sobe o servidor Ollama, baixa o modelo `qwen2.5:14b` e abre o túnel. O IA/Proxy Service se conecta a esse endpoint público (variável `NGROK_URL`) usando uma interface compatível com a API da OpenAI.

4. Fluxo de uma Requisição

A sequência abaixo descreve o caminho completo de uma pergunta, desde o clique do estudante até a resposta exibida na tela:

- 1 **Frontend → Gateway:** o estudante envia a pergunta e o `session_id` via `POST /perguntar`.
- 2 **Histórico:** o Gateway busca as últimas mensagens da sessão (memória) e salva a pergunta atual.
- 3 **Triagem de identidade:** se a pergunta é apenas sobre o nome da ARIA, responde de forma determinística e encerra o fluxo.
- 4 **Recuperação:** o Gateway limpa a consulta e chama `POST /contexto` no RAG, que devolve os trechos relevantes e os scores (cosseno e rerank).
- 5 **Roteamento:** com base no score, decide entre **material** (PDF) e **web**. Pedidos explícitos de busca na web forçam a rota externa.
- 6 **Busca web (se aplicável):** o Gateway aciona a ferramenta `buscar_web` no MCP, opcionalmente ancorando a consulta no tema do PDF.
- 7 **Geração:** o Gateway chama `POST /gerar` no IA/Proxy, que monta o prompt (com histórico e contexto) e consulta o LLM.
- 8 **Persistência e resposta:** a resposta é salva no histórico e devolvida ao frontend com a *origem* (material/web/identidade) e o score.

5. Pipeline de Recuperação (RAG)

5.1 Indexação em Pipeline (4 estágios paralelos)

Ao receber um PDF, o RAG executa uma indexação em **streaming** com quatro estágios que rodam em paralelo, conectados por filas. Enquanto um PDF é lido, outro trecho já está sendo fragmentado e um terceiro já está sendo vetorizado — maximizando o uso simultâneo de CPU e GPU:

- **Leitura** dos PDFs (I/O, em *threads*).
- **Chunking** (fragmentação) com `SentenceSplitter` (blocos de 768 tokens, sobreposição de 150).
- **Embedding** em lotes na GPU (e5-large).
- **Inserção** em lote no ChromaDB.

O índice é persistido em disco com um *hash* dos documentos; em reinícios, se o material não mudou, o índice é recarregado diretamente (startup rápido), sem reindexar.

5.2 Recuperação Híbrida + Reranking

A recuperação combina duas estratégias para maximizar o *recall* e, em seguida, refina a ordenação para maximizar a *precisão*:

- **Híbrido (recall):** um retriever vetorial (semântico) e um BM25 (lexical) recuperam até 20 candidatos, fundidos por *reciprocal rank fusion*.
- **Reranking (precisão):** um *cross-encoder* (bge-reranker-base) reordena esses candidatos por relevância real, mantendo apenas os 5 melhores, que vão ao LLM.
- **Calibração:** o reranker roda na CPU (não disputa a VRAM da GPU) e fornece um `rerank_score` (0–1) muito mais confiável que o cosseno cru para o roteamento.

5.3 Roteamento Material × Web

O Gateway decide a origem da resposta comparando o score de relevância a um limiar:

- Se o **rerank_score** estiver disponível, usa-o com limiar padrão 0,3 (sinal calibrado, preferido).
- Caso o reranker esteja indisponível, recai no **cosseno** com limiar 0,66 (fallback).
- $\text{Score} \geq \text{limiar}$ → resposta vem do **material** (PDFs). Abaixo → **busca na web**.
- Pedido explícito (“pesquise na web”) força a rota externa, ignorando o score.

5.4 Casos Especiais: Resumo, Questões e Identidade

Alguns pedidos são “meta” — referem-se ao documento como um todo, não a um ponto específico — e casam mal com a busca por similaridade. Para eles há caminhos dedicados:

- **Resumos e geração de questões:** em vez do top-k de uma consulta genérica, monta-se uma **amostra ampla** do conteúdo, distribuída do início ao fim do documento (trechos truncados), forçando a rota do material. Isso evita que o resumo “leia só o começo” do livro.
- **Identidade:** perguntas sobre o nome da ARIA são respondidas de forma determinística, sem recuperação, blindando o sistema contra confusão com o título/autor do PDF.

6. Camada de Geração (LLM)

A geração é centralizada no IA/Proxy Service, com forte ênfase em **controle de qualidade determinístico** em torno de um modelo de porte médio (14B), que tende a cometer deslizes previsíveis. As principais técnicas são:

- **Grounding rigoroso:** os prompts instruem o modelo a se basear estritamente no contexto fornecido e a declarar explicitamente quando a informação não está no material.
- **Precisão de dados:** números, datas, fórmulas e definições devem ser reproduzidos exatamente como aparecem no contexto.
- **Formatação didática:** uso de negrito para termos-chave, listas e subtítulos temáticos; proibição de citar números de página.
- **Identidade reforçada:** nome tratado fora do modelo; em perguntas combinadas, a saudação inventada pelo modelo é removida e a apresentação correta é recolada.
- **Pós-processamento de questões:** remoção de asteriscos desbalanceados, remoção opcional de gabarito e correção de concordância (“boa estudos” → “bons estudos”).
- **Janela de contexto fixa:** templates de resumo são propositalmente enxutos, pois cada token de instrução reduz o espaço disponível para a resposta no modelo de 8.192 tokens.

7. Decisões Técnicas e Lições Aprendidas

Ao longo do desenvolvimento, vários comportamentos do modelo e do ambiente exigiram soluções específicas. As mais relevantes estão consolidadas a seguir:

Desafio	Solução adotada
Modelo se diz “Claude/GPT” ao ser perguntado o nome	Identidade tratada de forma determinística (resposta fixa), fora do LLM
Consulta de resumo recupera a ficha em vez do conteúdo	Caminho dedicado com amostra ampla do documento (início→meio→fim)
Reranker devolve numpy.float32 e quebra a serialização JSON do FastAPI	Conversão explícita para float nativo antes de arredondar
pypdf trava o arquivo no Windows (WinError 32) ao reindexar	Leitura dos bytes para memória (BytesIO) e fechamento imediato do handle
Reindexar “por cima” mistura assuntos de livros diferentes	Política “um livro por vez” + limpeza da coleção do ChromaDB pela API
Modelo clona a mesma questão com outro número	Instruções de anti-repetição + amostra ampla do conteúdo
Asteriscos de negrito desbalanceados “vazam” nas alternativas	Remoção determinística de Markdown nas respostas de questões
Janela de contexto fixa do Qwen no Colab (T4)	Templates enxutos e contexto compacto para liberar espaço de saída
Colab/ngrok cai ou o túnel expira	Deteção de erros 403/timeout e mensagens HTTP 503 acionáveis ao estudante

8. Limitações e Trabalhos Futuros

- **Dependência do Colab/ngrok:** o LLM depende de uma sessão de Colab ativa e de um túnel que pode expirar; um *deploy* em servidor dedicado traria mais estabilidade.
- **Um material por vez:** a política atual indexa apenas um PDF; suportar múltiplos documentos exigiria isolamento por coleção ou por estudante.
- **Limiar de roteamento empírico:** os limiares de relevância foram calibrados manualmente e podem exigir reajuste para outros domínios de conteúdo.
- **Sem autenticação:** o sistema é voltado a uso local/acadêmico; um cenário de produção demandaria autenticação, autorização e isolamento de dados por usuário.
- **Modelo de porte médio:** o Qwen 14B exige diversos ajustes determinísticos; modelos maiores reduziriam a necessidade de pós-processamento.

9. Conclusão

O ARIA demonstra, na prática, como combinar **recuperação aumentada por geração (RAG)**, **busca híbrida com reranking** e um **LLM** em uma arquitetura de microserviços coesa e de baixo acoplamento. A separação clara de responsabilidades — orquestração, recuperação, geração, histórico e ferramentas externas — torna o sistema compreensível, testável e evolutivo.

Mais do que integrar tecnologias, o projeto evidencia um cuidado de engenharia com os **limites reais** de um modelo de porte médio e de um ambiente de execução gratuito: cada deslize previsível do modelo (identidade, alucinação, formatação, repetição) é contornado por uma salvaguarda determinística, e cada falha de infraestrutura é traduzida em uma mensagem clara ao usuário. O resultado é um tutor acadêmico que prioriza a **precisão** e a **experiência do estudante**, mantendo-se fiel ao material de aula.

— Fim do relatório —